Ruby has a featured called *refinements* that allow you to make these changes locally without affecting code outside the file you are writing. The goal of using refinements is to continue to allow you to make changes to existing classes, but to only allow those changes to be in effect when you explicitly say that they are. Not only does this mean that the changes are no longer global, it also means that when one of the added methods is used, there's an explicit statement in the file they are used in saying where added methods are coming from. This means a reader of the code has a better chance of finding the source more easily.

Now, before going any further, here is the mandatory warning. Refinements have been a part of Ruby since 2.0, but their actual uptake has been minor. It is unusual to see refinements in the wild. Even while it seems they solve a problem in theory, not very many teams seem to have found them to solve a problem in fact.

A refinement is a way of packaging a set of changes to one or more classes. These refinements are defined within a module. You can then elect to use this module with the refinements within a source file, in which case the change will apply to the source in that module past the point where the refinement is used. However, code outside this file is not affected. You can alternately elect to use the module with refinements inside a module or class, in which case the refinement applies inside that module or class, but not outside of it.

Let's make this concrete. Here's a vanity version of `puts` rewritten using refinements:

metaprogramming/vanity_puts.rb

```
module VanityPuts
  refine Object do
    private def puts(*args)
      args.each do |arg|
        Kernel.puts("Dave says: #{arg}")
      end
    end
  end
end
```

The refinement is contained in the module `VanityPuts`. The line `refine Object do` starts the definition. What's happening here is we are calling the method `Module#refine`, the receiver is the `VanityPuts` module. The `refine` method takes an argument, which is a class or module and a block. Within the block you define the methods that you would like to update in that class. At this point, no change is made to the underlying class—you have defined a method, but haven't yet told Ruby to use it.

That's what the `using` method does:

metaprogramming/use_in_class.rb

```
require_relative "vanity_puts"

class DavesStuff
  using VanityPuts

  def greet(msg)
    puts msg
  end
end

DavesStuff.new.greet("in DavesStuff")
```

*produces:*

```
Dave says: in DavesStuff
```